



TP3

Perceptrones

TP3 — Perceptrones
Versión resumida

Sistemas de Inteligencia Artificial · ITBA · 1Q 2026

Qué pide el TP

Tres ejercicios sobre perceptrones, todos evaluados experimentalmente.

Ej 1	Detección de fraude por <i>knowledge distillation</i> con perceptrón simple.
Ej 2	Clasificación de dígitos manuscritos (10 clases, 28×28).
Ej 3	Mismo problema, target: accuracy $\geq 98\%$ en test .

Regla de evaluación: `digits_test.csv` se evalúa *una sola vez*, al final, con el config ganador congelado. No se toca durante búsqueda de hiperparámetros.

Bloque 1 / 4

Decisiones de diseño

Siete decisiones que afectan a todos los ejercicios

Decisión	Ej 1 (perceptrón simple)	Ej 2 / Ej 3 (MLP)	Backing
1. Activación	sigmoide (target $[0, 1]$)	ReLU + softmax salida	literatura estándar
2. Inicialización	uniform $[-0,1, +0,1]$	<i>auto</i> : He (ReLU), Xavier (softmax)	HTML cátedra
3. Optimizador	SGD online	Adam ($lr=10^{-3}$)	PDF optimizadores
4. Learning rate	fijo (config)	10^{-3} (sweep)	PDF optimizadores
5. Función de costo	MSE	cross-entropy (multiclase)	literatura estándar
6. Convergencia	ϵ sobre MSE de train	early stopping val_loss (patience=10)	slide 14 PDF reg.
7. Bias	bias trick (col. de 1's)	bias trick + <i>no</i> se penaliza con L2	Goodfellow Cap. 7

Estrategia online vs. mini-batch: Ej 1 = online estricto (un update por muestra, regla del apunte).
Ej 2 / Ej 3 = mini-batch con **B=16** (elegido por sweep, mejor val_acc + más rápido).

Las elecciones críticas se validaron con sweep (Ej 2)

Coarse-to-fine: Fase 1 (k-fold=1, exploratoria) → `base.json` → Fase 2 (k-fold=5, validación).

HP	Ganador	Comentario
Optimizador	Adam (96.74 %)	Momentum 96.34 %, SGD 94.81 % (no convergió).
LR	10^{-3} (96.74 %)	10^{-4} lento; $\geq 10^{-2}$ overshooting; 0,1 y 10 diverge.
Arquitectura	[784, 100, 50, 10] (96.22 %)	Empate técnico con <code>wider</code> [200, 100]; elegida por parsimonia .
Init	empate técnico ($\sim 96\%$)	He / Xavier / Uniform iguales (std $\sim 0,3\%$).
Activación	ReLU (96.22 %)	Sigmoid 96.01 %, tanh 95.94 %; ReLU converge $\sim 3\times$ más rápido.
Batch	16 (mejor macro_F1)	32/64/128 empatan en val_acc, ≥ 7 epochs vs. 3.

Solo el sweep de LR mostró rangos catastróficos. Las demás dimensiones rinden parecido $\Rightarrow$ elegimos por **parsimonia** (más simple / más rápido a igual rendimiento).

Bloque 2 / 4

Ej 1 — Detección de fraude

Ej 1 — El problema

Knowledge distillation: replicar la salida del BigModel (probabilidad de fraude) con un TinyModel (perceptrón simple).

Datos: `fraud_dataset.csv` (9 features, $\sim 50k$ filas).

Target de entrenamiento:

`big_model_fraud_probability` (continuo en $[0, 1]$).

Ground truth (eval):

`flagged_fraud` (binaria 0/1).

Regla del enunciado:

`flagged_fraud` *nunca* se usa para entrenar.

Salida esperada en $[0, 1]$ $\Rightarrow$
activación sigmoide en TinyModel.

Ej 1 — Lineal vs No-Lineal: la sigmoide gana claro

Modelo	MSE test (mean $\pm$ std)
Lineal (Adaline)	0,0266 $\pm$ 0,0008
No-Lineal (sigmoide)	0,0110 $\pm$ 0,0004 (−59 %)

El target es una probabilidad acotada en $[0, 1]$. La sigmoide *naturalmente* mapea su salida a ese rango; el lineal puede producir valores fuera de $[0, 1]$ y el MSE penaliza fuerte esa salida fuera de dominio.

Ej 1 — Threshold óptimo y recomendación

Threshold default 0.5 luce terrible (precision 0.333, recall 1.000). No es el modelo: las salidas sigmoide se concentran en la parte alta (mediana de scores: 0.99).

Threshold	Precision	Recall	F1	Caso de uso
0.50 (default)	0.333	1.000	0.499	—
0.89 (óptimo F1)	0.886	0.861	0.873	auditoría manual barata
0.95	0.949	0.746	0.835	FP costosos
0.99	1.000	0.527	0.690	cero tolerancia a FP

AUC-ROC = 0.9921 (lineal: 0.9720) — capacidad de *ranking* excelente.

El modelo entrega scores; la política de threshold es decisión de negocio. Recomendación: 0,89 si los falsos positivos son baratos; $\geq 0,95$ si son costosos.

Bloque 3 / 4

Ej 2 — Clasificación de dígitos

Ej 2 — El problema y la arquitectura final

Datos:

- Train: `digits.csv` (12.449 muestras)
- Test: `digits_test.csv` (2.498)

Cada muestra: vector 784 floats $\in [0, 1]$
(28×28 flatten), label 0–9.

Arquitectura final:

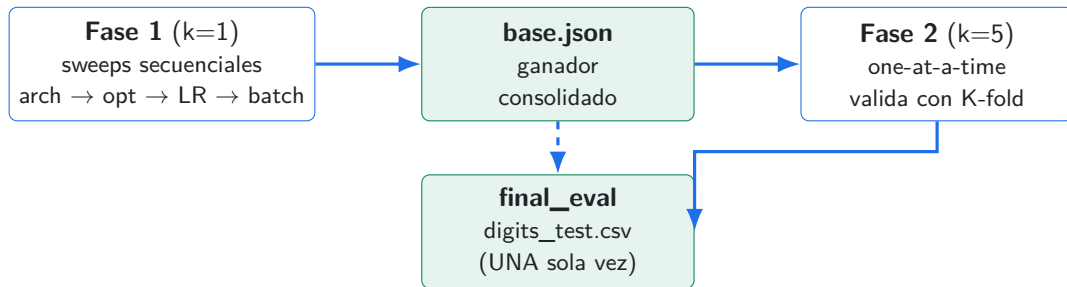
- Input: 784
- Hidden 1: 100 (ReLU)
- Hidden 2: 50 (ReLU)
- Output: 10 (Softmax)

Loss: cross-entropy. Adam ($lr = 10^{-3}$).

Batch=16. Early stopping (patience=10).

Regla: `digits_test.csv` se evalúa **una sola vez** al final.

Ej 2 — La experimentación, en una slide



Coarse-to-fine: Fase 1 reduce el espacio de búsqueda con sweeps rápidos. Fase 2 valida con K-fold=5.

⇒ La metodología de Fase 1 ya hace el grueso; Fase 2 confirma que las elecciones son robustas, no suerte de un solo split.

Ej 2 — Resultados: val K-fold vs. test (el cachetazo)

val K-fold=5: **96.22 % $\pm$ 0.43 %**

↓ (final_eval contra digits_test.csv, UNA sola vez)

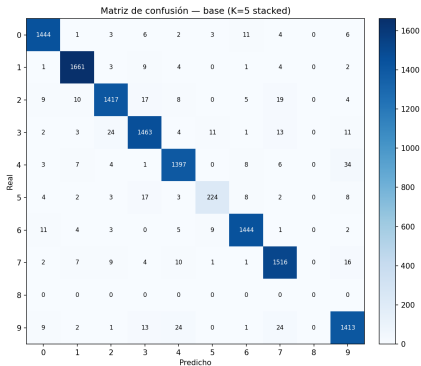
test: **86.30 %**

Drop de 10 puntos porcentuales.

Distribution shift, no underfitting. El modelo está bien — los datos no se parecen. Diez veces más grande que el std del K-fold (0.43 %). Imposible que sea ruido.

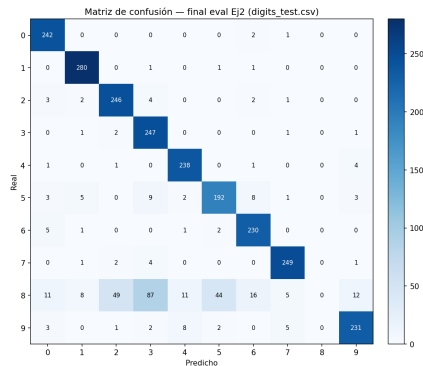
Ej 2 — La matriz de confusión confirma el shift

Val K-fold (digits.csv)



Diagonal limpia para 9 clases. Clase 8 colapsa.

Test (digits_test.csv)



Confusiones distribuidas en todo el grid.

Bloque 4 / 4

Ej 3 — Mejorar el accuracy

Ej 3 — Plan y el primer dominó: más datos

Hipótesis: el techo del Ej 2 es por shift, no falta de capacidad.

Plan secuencial: (1) más datos → (2) Pack C (regularización) → (3) capacidad si nada llega.

val K=5: 96,22 % → 97,06 %

test: 86,30 % → 96,36 %

+10 puntos porcentuales en test.

Cero cambios al modelo. Solo `more_digits.csv` (15.742 muestras extras).

Macro F1: 0,857 → 0,959. La clase 8, que estaba colapsada en Ej 2, ahora se predice correctamente.

TP3 → Hipótesis del shift confirmada. Pero faltan ~ 1,6 pp para llegar al 98 %.

Ej 3 — Pack C ↔ clase de regularización

¿Qué dice el PDF de regularización y qué implementamos?

Técnica del PDF	Impl.	Resultado en Ej 3
Early stopping (slide 14)	✓	Activo en todos los configs
Augmentation gaussiano (slide 18)	✓	$\sigma = 0,05$, +0,32 pp (parte del ganador)
Augmentation rotaciones / traslaciones / escalas (slide 18)	✗	—
L2 / Weight Decay (slides 20–25)	✓	$\lambda = 10^{-4}$, +0,20 pp
Dropout (slide 26 — mención)	✓	val sí, test no (−0,08 pp)
Modelos de ensamble / semi-supervisado / adversarial	—	fuera de alcance

Lo que faltó: las tres formas de augmentation *geométrica* (rotaciones, traslaciones, escalas). Es la hipótesis principal del techo en 96.88 %.

Ej 3 — Resultados Pack C: el ganador

Config	val K=5	test	Δ vs base_extra
base_extra_data	0.9706	0.9636	— (referencia)
+ L2 (1e-4)	0.9758	0.9656	+0,20 pp
+ dropout (p=0.2)	0.9750	0.9628	−0,08 pp
+ L2 + aug ($\sigma = 0,05$)	0.9760	0.9688	+0,52 pp
+ L2 + aug + dropout	0.9768	0.9656	+0,20 pp
+ wider + L2 + aug	0.9777	0.9640	+0,04 pp

Ganador: L2 + augmentation gaussiana ($\sigma = 0,05$) → **96.88 % en test**.

Dropout no transfirió: gana en val, pierde en test (regulariza dentro de la distribución del train, no compensa el shift). **Wider mejora val, empeora test** ⇒ descarta falta de capacidad.

Ej 3 — Por qué no llegamos al 98 %

Mejor: 96.88 %. Faltan 1.12 pp.

Tres hipótesis ordenadas por probabilidad:

1. **Augmentation insuficiente.** Gaussian noise es *isotrópico* (por píxel). El shift parece ser *geométrico*: rotación, traslación, grosor de trazo. Pide augmentation **geométrica**, que el slide 18 del PDF lista pero **no implementamos**.
2. **Sub-representación en val interno del final_eval.** Val 10 % random para early stopping puede no incluir los digits difíciles del test.
3. **Capacidad:** improbable, *wider* sobreajustó (consistente con la curva U del slide 9 PDF regularización: más capacidad sin más datos $\Rightarrow$ overfitting).

Próximo paso natural: augmentation por rotación / traslación pequeñas. No implementada en este TP.

Conclusiones

Tres ideas para llevarse:

1. Una buena pipeline gana más que tunear hiperparámetros.

El +10 pp del Ej 3 vino de *datos*, no de modelo. La regularización aportó solo +0,5 pp adicional.

2. Los empates técnicos son más comunes que los wins claros.

Mayoría de los sweeps con margen $< 0,3$ pp y std $\sim 0,4\%$. Hay que elegir por **parsimonia**, no por 0.05 pp en val.

3. “Mejor en val” $\neq$ “mejor en test” cuando hay shift.

Dropout fue el ejemplo de manual: ganador en val, perdedor en test.

Item	Status
Ej 1 — Knowledge distillation + threshold sweep	✓
Ej 2 — sweeps + final_eval + análisis del shift	✓
Ej 3 — more_digits.csv + Pack C + análisis	✓
Ej 3 — target $\geq 98\%$	✗ (96.88 %)

Gracias.

Preguntas.
